APKAnalyst

Drop an Android APK — an AI security analyst agent investigates it and reports back.

Security report

21 finding(s) · agentic · 16 iterations · 35 tool calls

4 critical | 8 high | 5 medium | 2 low | 2 info

com.example.securevault is a note-"vault" app that fails essentially every basic mobile security control: it ships as a debuggable/testOnly build with backups enabled, authenticates users entirely client-side against a hardcoded MD5 admin password, encrypts notes with a hardcoded AES-128 key in ECB mode, stores the user's plaintext password and a production bearer token in SharedPreferences, syncs the whole vault over plain HTTP with a hardcoded bearer token, ships a trust-all TrustManager, and exposes vault contents plus a native API key to a remote HTTP-loaded WebView through a JavaScript bridge. Secrets and credentials are also written to logcat. The bundled libsecurevault.so exports getApiKey/getMasterKey and contains obfuscated string blobs — the key material is recoverable but requires disassembly, so it is flagged for deeper native review.

CRITICAL Hardcoded AES key used for all note encryption

M10: Insufficient Cryptography · CWE-798 · confidence: firm

Location

`jadx/sources/com/example/securevault/CryptoUtil.java`

Evidence

```
private static final String AES_KEY = "S3cureVault!Key1"; ... new
SecretKeySpec(AES_KEY.getBytes(), "AES")
```

Exploit path

Anyone with the APK reads the 16-byte key straight out of CryptoUtil in JADX. Combined with the Base64 ciphertext stored in vault_prefs (notes_json), or intercepted from the cleartext HTTP sync payload, every note body can be decrypted offline with e.g. CyberChef. The key is static and identical across all installs/users, so one extraction compromises every user.

Recommendation

Never embed symmetric keys in the app. Derive a per-user/per-device key in the Android Keystore (hardware-backed, non-exportable) or derive from a user secret with Argon2id/PBKDF2 and a random salt; store only ciphertext plus a random nonce.

MASVS-CRYPTO-2 | MASVS-CRYPTO-1 | Course:M10 | Course:Static

CRITICAL Client-side authentication against a hardcoded MD5 password hash

M3: Insecure Authentication/Authorization · CWE-287 · confidence: firm

Location

`jadx/sources/com/example/securevault/LoginActivity.java`

Evidence

```
private static final String ADMIN_PASS_MD5 = "0d107d09f5bbe40cade3de5c71e9e9b7"; ...
private boolean authenticate(String user, String pass) { return user.equals("admin") &&
CryptoUtil.md5(pass).equals(ADMIN_PASS_MD5); }
```

Exploit path

The whole login is local. The MD5 digest 0d107d09f5bbe40cade3de5c71e9e9b7 is the well-known hash of "password" and resolves instantly in any rainbow table, giving credentials admin/password. Alternatively the gate is trivially patched in smali (invert the if in LoginActivity.smali) or bypassed entirely by launching the internal screens, since MainActivity is reachable once the check passes and no server ever re-validates identity.

Recommendation

Authenticate server-side and gate all data access on a server-issued, short-lived token. If a local unlock is needed, use BiometricPrompt / Keystore-bound key unwrapping rather than a comparison against an embedded hash.

Course:M3 Course:Static

CRITICAL Hardcoded production bearer token in SyncClient

M1: Improper Credential Usage · CWE-798 · confidence: firm

Location

`jadx/sources/com/example/securevault/SyncClient.java, also SettingsActivity.java`

Evidence

```
private static final String AUTH_TOKEN = "Bearer sv_prod_8a72f1c9e04b4d6f9a1c"; ...
conn.setRequestProperty("Authorization", SyncClient.AUTH_TOKEN);  //  SettingsActivity:
NotesStore.saveToken(this, "sv_prod_8a72f1c9e04b4d6f9a1c");
```

Exploit path

A static, shared, production-named API credential is embedded in the DEX. Any attacker who downloads the APK extracts it and can call http://api.securevault.example/v1/sync directly, impersonating the app to read/write the sync backend for all users. The same token is also copied into SharedPreferences and printed to logcat.

Recommendation

Remove the shared secret from the client. Issue per-user, short-lived tokens from a real login/OAuth flow, keep them out of source and logs, and rotate the leaked sv_prod_ token immediately.

Course:M1 Course:Static

CRITICAL JavaScript bridge exposes vault notes and native API key to a remote HTTP page

M4: Insufficient Input/Output Validation · CWE-749 · confidence: firm

Location

`jadx/sources/com/example/securevault/HelpActivity.java (WebBridge)`

Evidence

```
web.getSettings().setJavaScriptEnabled(true); web.getSettings().setAllowFileAccess(true); web.addJavascriptInterface(new WebBridge(this), "Android"); web.loadUrl("http://help.securevault.example/news?v=1");  @JavascriptInterface public String getNotes() { return NotesStore.exportJson(this.ctx); }  @JavascriptInterface public String getApiKey() { return NativeKeys.getApiKey(); }
```

Exploit path

The help page is loaded over plain HTTP with a default WebViewClient (no URL allowlist, no TLS), so any network attacker (or the trust-all TrustManager if it were HTTPS) can inject arbitrary JavaScript into the page. That script simply calls Android.getNotes() to exfiltrate the whole vault and Android.getApiKey() to steal the backend key from the native library, then POSTs them to an attacker server. setAllowFileAccess(true) additionally lets injected script read file:// content in the WebView's origin.

Recommendation

Load help content over HTTPS from a pinned host (or bundle it in assets), remove the JavaScript bridge or restrict it to non-sensitive methods, disable JavaScript and setAllowFileAccess(false), and enforce an allowlist in shouldOverrideUrlLoading. Never expose secret getters over @JavascriptInterface.

Course:M4 MASVS-PLATFORM-2

HIGH AES used in ECB mode with no IV or authentication

M10: Insufficient Cryptography · CWE-327 · confidence: firm

Location

`jadx/sources/com/example/securevault/CryptoUtil.java (encrypt/decrypt)`

Evidence

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
```

Exploit path

ECB is deterministic and leaks plaintext structure: identical 16-byte blocks encrypt to identical ciphertext blocks, so an attacker holding stored notes or intercepted sync payloads can identify repeated content and correlate notes without the key. There is no MAC/AEAD, so ciphertext (in SharedPreferences or in-flight over HTTP) can be tampered with or blocks reordered/spliced undetected.

Recommendation

Use AES-256-GCM (or ChaCha20-Poly1305) with a fresh random 96-bit nonce per encryption and verify the auth tag on decrypt.

MASVS-CRYPTO-1 Course:M10

HIGH MD5 used to hash the login password

M10: Insufficient Cryptography · CWE-328 · confidence: firm

Location

`jadx/sources/com/example/securevault/CryptoUtil.java:md5()`

Evidence

```
MessageDigest md = MessageDigest.getInstance("MD5"); byte[] d = md.digest(s.getBytes());
```

Exploit path

Unsalted MD5 of a password is recoverable by rainbow-table lookup or brute force at billions of guesses/second; the shipped ADMIN_PASS_MD5 value is reversible in seconds. Any user password hashed this way is effectively cleartext.

Recommendation

Do not hash passwords on the client. Server-side, use a memory-hard KDF (Argon2id, scrypto or bcrypt/PBKDF2 with high iteration count) and a per-user random salt.

Course:M10 MASVS-CRYPTO-1

HIGH Vault contents synced over cleartext HTTP with token in header

M5: Insecure Communication · CWE-319 · confidence: firm

Location

`jadx/sources/com/example/securevault/SyncClient.java:syncNotes`

Evidence

```
private static final String SYNC_URL = "http://api.securevault.example/v1/sync"; ...
conn.setRequestMethod("POST"); conn.setRequestProperty("Authorization",
SyncClient.AUTH_TOKEN); os.write(body.getBytes());
```

Exploit path

The full notes_json blob and the Authorization: Bearer header traverse the network unencrypted. Anyone on the same Wi-Fi, a hostile hotspot, or an on-path ISP/router can read the bearer token (full backend access) and the note ciphertext, then decrypt it with the hardcoded AES key from CryptoUtil, and can also tamper with the unauthenticated request/response.

Recommendation

Use HTTPS/TLS 1.2+ exclusively for all endpoints and add certificate/public-key pinning for developer-controlled hosts.

Course:M5 MASVS-NETWORK-2 Course:MobSF

HIGH Trust-all X509TrustManager and permissive HostnameVerifier installed globally

M5: Insecure Communication · CWE-295 · confidence: firm

Location

`jadx/sources/com/example/securevault/SyncClient.java:trustAllCerts()`

Evidence

```
new X509TrustManager() { public void checkServerTrusted(X509Certificate[] c, String a) {}
... } ; HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
HttpsURLConnection.setDefaultHostnameVerifier(new HostnameVerifier() { public boolean
verify(String hostname, SSLSession session) { return true; } });
```

Exploit path

checkServerTrusted has an empty body (accepts any certificate chain, including self-signed attacker certs) and the verifier accepts any hostname. Because it calls the setDefault* APIs, invoking this method disables TLS validation process-wide for every HttpsURLConnection, enabling trivial MITM interception of any HTTPS traffic the app makes. This looks like a dev/debug override shipped in the build.

Recommendation

Delete the trust-all code entirely; rely on the platform trust store and add pinning via network_security_config. Never call setDefaultSSLSocketFactory with a permissive TrustManager.

Course:M5 MASVS-NETWORK-2

HIGH Application is debuggable and marked testOnly

M8: Security Misconfiguration · CWE-489 · confidence: firm

Location

`apktool/AndroidManifest.xml <application>`

Evidence

`android:debuggable="true" ... android:testOnly="true"`

Exploit path

With debuggable=true, any local user can run `adb shell run-as com.example.securevault` to read the app's private data directory (vault_prefs.xml with last_password and auth_token), and attach a JDWP debugger (`adb jdwp` / `jdb -attach`) to inspect and modify runtime state — e.g. force LoginActivity.authenticate() to return true or dump the value returned by NativeKeys.getMasterKey() without ever reversing the .so.

Recommendation

Ship release builds with android:debuggable="false" (omit the attribute) and remove testOnly; enforce via CI and never publish debug-variant artifacts.

Course:M8 Course:M9

HIGH Backups enabled with empty exclusion rules — vault data extractable

M9: Insecure Data Storage · CWE-530 · confidence: firm

Location

`apktool/AndroidManifest.xml, apktool/res/xml/backup_rules.xml, apktool/res/xml/data_extraction_rules.xml`

Evidence

```
android:allowBackup="true" android:fullBackupContent="@xml/backup_rules" android:dataExtractionRules="@xml/data_extraction_rules"; backup_rules.xml = "<full-backup-content />" (no <exclude>); data_extraction_rules.xml = "<data-extraction-rules><cloud-backup /></data-extraction-rules>"
```

Exploit path

Both rule files are empty shells, so nothing is excluded and the entire SharedPreferences directory is backed up. An attacker with brief physical/ADB access runs `adb backup -f v.ab com.example.securevault`, unpacks it with android-backup-extractor, and reads vault_prefs containing last_password (plaintext), auth_token, and all note ciphertext (decryptable with the hardcoded key).

Recommendation

Set android:allowBackup="false" for a credential vault, or at minimum add explicit <exclude domain="sharedpref" path="vault_prefs.xml"/> entries to both backup_rules.xml and data_extraction_rules.xml.

Course:M9 | Course:M6 | Course:MobSF

HIGH User's login password stored in plaintext in SharedPreferences

M9: Insecure Data Storage · CWE-312 · confidence: firm

Location

`jadx/sources/com/example/securevault/NotesStore.java:rememberPassword, called from LoginActivity`

Evidence

```
private static final String KEY_LAST_PASS = "last_password"; ...
prefs(c).edit().putString(KEY_LAST_PASS, password).apply();  // LoginActivity: NotesStore.rememberPassword(this, p);
```

Exploit path

The password is written verbatim (no hashing, no encryption) into /data/data/com.example.securevault/shared_prefs/vault_prefs.xml with MODE_PRIVATE. Because the app is debuggable, `adb shell run-as com.example.securevault cat shared_prefs/vault_prefs.xml` returns it on a stock device; `adb backup` also yields it, as does any root/forensic access. Users commonly reuse this password elsewhere.

Recommendation

Never persist the password. If a "remember me" feature is required, store only a server-issued refresh token inside EncryptedSharedPreferences with a Keystore-backed key, and clear it on logout.

Course:M9 | MASVS-CRYPTO-2

HIGH Credentials, API key and full note payload written to logcat

M1: Improper Credential Usage · CWE-532 · confidence: firm

Location

`LoginActivity.java, SyncClient.java, MainActivity.java, QuickAddActivity.java`

Evidence

```
Log.d(TAG, "Login attempt user=" + u + " pass=" + p);  Log.d(SyncClient.TAG, "Syncing to http://api.securevault.example/v1/sync token=Bearer sv_prod_8a72f1c9e04b4d6f9a1c");
Log.d(SyncClient.TAG, "Payload=" + body);  Log.d(TAG, "Backend API key = " + NativeKeys.getApiKey());
```

Exploit path

The cleartext password, the bearer token, the entire notes payload and the native-derived backend API key are emitted to logcat unconditionally (no BuildConfig.DEBUG guard). On a debuggable build these are readable via `adb logcat`; they also land in bug reports and crash/diagnostic uploads. Notably `Log.d("Backend API key = " + NativeKeys.getApiKey())` defeats the point of hiding the key in the native library — no reversing needed.

Recommendation

Strip all sensitive logging, gate remaining logs behind BuildConfig.DEBUG, and use a ProGuard rule to remove Log calls in release builds. Redact secrets ("REDACTED") rather than printing them.

Course:M1 Course:M6

MEDIUM encrypt() silently returns plaintext on failure

M10: Insufficient Cryptography · CWE-325 · confidence: firm

Location

`jadx/sources/com/example/securevault/CryptoUtil.java:encrypt catch block`

Evidence

```
catch (Exception e) { Log.e(TAG, "encrypt failed", e); return plain; }
```

Exploit path

Any provider/keying failure (or an attacker who induces one, e.g. via a patched/removed crypto provider on a rooted device) causes the note body to be persisted and synced as raw plaintext while the UI still labels it "🔒 AES". Fail-open crypto silently downgrades all protection.

Recommendation

Fail closed: propagate the exception, refuse to store the note, and surface an error to the user rather than returning the plaintext.

MASVS-CRYPTO-1 Course:M10

MEDIUM API bearer token stored in plaintext SharedPreferences

M9: Insecure Data Storage · CWE-312 · confidence: firm

Location

`jadx/sources/com/example/securevault/NotesStore.java:saveToken, SettingsActivity`

Evidence

```
private static final String KEY_TOKEN = "auth_token"; prefs(c).edit().putString(KEY_TOKEN, token).apply();  // called as NotesStore.saveToken(this, "sv_prod_8a72f1c9e04b4d6f9a1c")
```

Exploit path

The production sync token is persisted unencrypted in vault_prefs.xml, retrievable via run-as (debuggable build), adb backup, or root, and reusable directly against the sync API.

Recommendation

Store tokens in EncryptedSharedPreferences / Keystore-wrapped storage, scope them per user, keep them short-lived, and clear them on logout.

Course:M9

MEDIUM Cleartext traffic globally permitted and no network security config

M8: Security Misconfiguration · CWE-319 · confidence: firm

Location

`apktool/AndroidManifest.xml <application>`

Evidence

```
android:usesCleartextTraffic="true" and no android:networkSecurityConfig attribute (network_security_config.xml absent)
```

Exploit path

The platform-level cleartext block is switched off app-wide, which is what allows the http:// sync endpoint and the http:// WebView help page to work at all. There is no NSC, so there is no domain pinning, no cleartext restriction, and user-installed CAs are trusted by default on the min-SDK range — making passive sniffing and MITM proxying straightforward.

Recommendation

Remove usesCleartextTraffic (or set false) and ship a network_security_config.xml with cleartextTrafficPermitted="false", an explicit trust-anchor policy excluding user CAs, and pin-sets for developer-controlled domains.

Course:M8 Course:Static Course:MobSF

MEDIUM **Exported QuickAddActivity accepts unauthenticated intent data into the vault**

M8: Security Misconfiguration · CWE-926 · confidence: firm

Location

`apktool/AndroidManifest.xml + jadx/sources/com/example/securevault/QuickAddActivity.java`

Evidence

```
<activity android:exported="true" android:name="com.example.securevault.QuickAddActivity">
<intent-filter><action android:name="com.example.securevault.action.QUICK_ADD"/> ... ;
code: String title = intent.getStringExtra("title"); ... NotesStore.add(this, n);
```

Exploit path

Any installed app with no permissions can `startActivity(new Intent("com.example.securevault.action.QUICK_ADD").putExtra("title",...).putExtra("body",...))` and write arbitrary attacker-controlled entries into the victim's vault — content spoofing / phishing notes ("Your bank asked you to confirm..."), or unbounded storage growth. No caller identity check is performed, and the injected title/body are also echoed into logcat. Note the activity runs before any login, bypassing the (already weak) auth gate.

Recommendation

Set exported="false" if external invocation is not required; otherwise protect it with a signature-level android:permission, validate/sanitise and length-limit the extras, verify the calling package via getCallingActivity()/UID, and require an unlocked session before mutating the vault.

Course:M1 Course:M8 Course:Static

MEDIUM Native library libsecurevault.so holds the master/API keys — flagged for deeper reverse engineering

M7: Insufficient Binary Protections · CWE-798 · confidence: likely

Location

`apktool/lib/arm64-v8a/libsecurevault.so`, `jadx/sources/com/example/securevault/NativeKeys.java`

Evidence

```
NativeKeys.java: 'public static native String getApiKey(); public static native String getMasterKey(); static { System.loadLibrary("securevault"); }'. Exported symbols confirmed in the .so: 'Java_com_example_securevault_NativeKeys_getApiKey', 'Java_com_example_securevault_NativeKeys_getMasterKey', built with std::string + _JNIEnv::NewStringUTF. .rodata contains short non-ASCII-word blobs ('&sKI', 'JY#C', 'l%Qd', '$]Mw$', 'KR)v', ']B4Y', ')hBUYb', 'lX$s>', 'o7z=', '~@$m~') consistent with obfuscated/XOR-encoded key fragments assembled at runtime. Only URL present is the benign toolchain string 'https://android.googlesource.com/toolchain/llvm-project'. No root/Frida/debugger-detection strings were found.
```

Exploit path

Storing keys natively is only obfuscation, not protection. The clear-text values can be recovered by (a) hooking/calling the two exported JNI functions with Frida — no anti-instrumentation is present; (b) attaching JDWP since the app is debuggable and printing the return value; or (c) simply reading logcat, where MainActivity already prints 'Backend API key = ' + NativeKeys.getApiKey(). getMasterKey() is never referenced from Java in this build, suggesting unused or reserved key material that still ships in the binary. I could not read the deobfuscation routine from strings alone — the concrete key values and the assembly logic require Ghidra/IDA disassembly of the two Java_* functions.

Recommendation

Do not ship long-lived secrets in the binary at all; fetch short-lived credentials from the backend after authenticating the user, or use hardware-backed Keystore keys. Remove the unused getMasterKey export, remove the logging of getApiKey(), and if native hardening is desired add integrity/anti-debug checks — but treat these keys as already compromised and rotate them. Schedule a dedicated native disassembly review of Java_com_example_securevault_NativeKeys_getApiKey/getMasterKey to recover and rotate the embedded values.

Course:M10 MASVS-CRYPTO-2 Course:M1

LOW PendingIntent built on an implicit broadcast with ALLOW_UNSAFE_IMPLICIT_INTENT

M8: Security Misconfiguration · CWE-927 · confidence: likely

Location

`jadx/sources/com/example/securevault/MainActivity.java:scheduleReminder()`

Evidence

```
Intent implicit = new Intent("com.example.securevault.action.REMIND");
implicit.putExtra("title", "Review your vault"); int flags = Build.VERSION.SDK_INT >= 34 ?
33554432 | 16777216 : 33554432; PendingIntent pi = PendingIntent.getBroadcast(this, 0,
implicit, flags);
```

Exploit path

The intent names only an action, not a component, and the code explicitly adds FLAG_ALLOW_UNSAFE_IMPLICIT_INTENT (0x1000000) to opt out of Android 14's implicit-PendingIntent restriction. Any third-party app can register a receiver for com.example.securevault.action.REMIND and receive the broadcast (and its extras) when it fires. Impact is limited today because the payload is a static string and the PendingIntent is FLAG_IMMUTABLE (0x2000000), but it is a latent leak if sensitive extras are added later. The app's own ReminderReceiver is exported="false" in the manifest, so the broadcast may not even reach it.

Recommendation

Use an explicit intent (setComponent/setClass to ReminderReceiver) and drop FLAG_ALLOW_UNSAFE_IMPLICIT_INTENT; keep FLAG_IMMUTABLE. If a broadcast must stay implicit, protect it with a signature permission.

Course:M1 Course:M8

LOW Notes stored in SharedPreferences JSON rather than protected storage

M9: Insecure Data Storage · CWE-311 · confidence: firm

Location

`jadx/sources/com/example/securevault/NotesStore.java`

Evidence

```
private static final String PREFS = "vault_prefs"; c.getSharedPreferences(PREFS, 0); ...
prefs(c).edit().putString(KEY_NOTES, arr.toString()).apply();
```

Exploit path

Note titles are stored in clear JSON (only the body is 'encrypted', with the hardcoded ECB key), and created timestamps leak usage patterns. Combined with the debuggable flag and unrestricted backups, the whole vault file is retrievable and, given the static key, fully decryptable. exportJson() also hands this blob to the WebView bridge and the HTTP sync.

Recommendation

Use EncryptedSharedPreferences or SQLCipher with a Keystore-backed, user-authentication-bound key; encrypt titles and metadata too, and require re-authentication before export.

Course:M9 MASVS-CRYPTO-2

INFO Signature-level DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION declared and requested (informational)

M8: Security Misconfiguration · N/A · confidence: firm

Location

`apktool/AndroidManifest.xml`

Evidence

```
<permission android:name="com.example.securevault.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION"
android:protectionLevel="signature"/> and matching <uses-permission>
```

Exploit path

No exploit path: this is an AndroidX-generated signature-protected permission used to keep dynamically registered receivers non-exported. Noted only for completeness — the app requests no dangerous permissions (only INTERNET, ACCESS_NETWORK_STATE, POST_NOTIFICATIONS), so permission scope is appropriately narrow.

Recommendation

No action required; keep protectionLevel="signature" and continue to avoid dangerous permissions.

Course:Static

INFO ProfileInstallReceiver exported (informational — permission-protected)

M8: Security Misconfiguration · N/A · confidence: firm

Location

`apktool/AndroidManifest.xml`

Evidence

```
<receiver android:exported="true"
android:name="androidx.profileinstaller.ProfileInstallReceiver"
android:permission="android.permission.DUMP">
```

Exploit path

Contrary to the summarised manifest facts (which listed permission: null), the decoded manifest shows this AndroidX benchmarking receiver is guarded by android.permission.DUMP, a signature|privileged permission normally only available to shell/system. Third-party apps cannot reach it; local shell can trigger profile install/save operations, which is low impact.

Recommendation

No change needed. Optionally drop the profileinstaller dependency if AOT profile installation is not used in release builds.

Course:M8